Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н. Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н. Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления» (ИУ)

КАФЕДРА «Информационная безопасность» (ИУ8)

ЛАБОРАТОРНАЯ РАБОТА № 8

Радиально-базисная нейронная сеть (RBF)

Вариант 2

Студент ИУ8-13М (Группа) Бочкарев Михаил Александрович (И. О. Фамилия) _____ (Подпись, дата)

Преподаватель Басараб Михаил Алексеевич (И. О. Фамилия) _____ (Подпись, дата)

Оценка: _____

Москва, 2026 г.

СОДЕРЖАНИЕ

1 Цель работы и постановка задачи 3
1.1 Таблица истинности 3
2 Модель радиально-базисной сети 5
2.1 Выбор центров и вычисление базисных функций 5
2.2 Выход сети 5
3 Алгоритм обучения и поиска выборки 7
3.1 Коррекция выходных весов 7
3.2 Перебор обучающих подмножеств 8
4 Результаты численного эксперимента 9
4.1 Первое обновление 9
4.2 Обучение на полной таблице 9
4.3 Минимальная найденная выборка 10
4.4 Выходные векторы и сравнение экспериментов 11
5 Программная реализация 13
6 Выводы 14
ПРИЛОЖЕНИЕ А Исходный код программы 15

1 Цель работы и постановка задачи

Цель работы — реализовать радиально-базисную нейронную сеть для воспроизведения булевой функции, исследовать обучение выходного слоя и найти минимальную обучающую выборку при фиксированных центрах радиально-базисных функций.

Для варианта 2 задана функция четырёх булевых переменных

$$F(x_1, x_2, x_3, x_4) = \overline{(x_3 \vee x_4)} \wedge \overline{x_1} \vee x_2. \tag{1}$$

Эквивалентная запись с явной группировкой операций:

$$F = \big(\neg x_3 \wedge \neg x_4 \wedge \neg x_1\big) \vee x_2.$$

Входной вектор $\mathbf{x} = (x_1, x_2, x_3, x_4)$ принимает все $2^4 = 16$ двоичных комбинаций, целевое значение $t = F(\mathbf{x})$ принадлежит множеству $\{0, 1\}$.

В работе выполняются два эксперимента: обучение на полной таблице истинности и поиск наименьшего подмножества для обучения выходных весов. В обоих случаях правильность проверяется на всех 16 комбинациях. Центры сети определяются один раз по полной таблице и при поиске подмножества не изменяются.

1.1 Таблица истинности

Образцы упорядочены лексикографически: от 0000 до 1111. Первой переменной соответствует старший разряд. Таблица 1 задаёт обучающие и проверочные данные.

Таблица 1 – Таблица истинности заданной функции

№	x_1	x_2	x_3	x_4	t
1	0	0	0	0	1
2	0	0	0	1	0
3	0	0	1	0	0
4	0	0	1	1	0
5	0	1	0	0	1
6	0	1	0	1	1
7	0	1	1	0	1
8	0	1	1	1	1
9	1	0	0	0	0

№	x_1	x_2	x_3	x_4	t
10	1	0	0	1	0
11	1	0	1	0	0
12	1	0	1	1	0
13	1	1	0	0	1
14	1	1	0	1	1
15	1	1	1	0	1
16	1	1	1	1	1

Если $x_2 = 1$, функция равна единице независимо от остальных переменных: это восемь образцов. При $x_2 = 0$ единица получается только для $\mathbf{x} = (0, 0, 0, 0)$. Поэтому число нулевых и единичных значений составляет

$$J_0 = 7, \qquad J_1 = 9.$$

2 Модель радиально-базисной сети

2.1 Выбор центров и вычисление базисных функций

Количество радиально-базисных нейронов выбирается по меньшему классу:

$$J = \min(J_0, J_1) = 7. \tag{2}$$

В качестве центров $\mathbf{c}_j$ используются все входные векторы класса 0, перечисленные в таблице 2.

Таблица 2 – Центры радиально-базисных функций

j	c_{j1}	c_{j2}	c_{j3}	c_{j4}
1	0	0	0	1
2	0	0	1	0
3	0	0	1	1
4	1	0	0	0
5	1	0	0	1
6	1	0	1	0
7	1	0	1	1

Для каждого центра вычисляется гауссова радиально-базисная функция

$$\varphi_j(\mathbf{x}) = \exp\left(-\sum_{i=1}^{4}(x_i - c_{ji})^2\right). \tag{3}$$

Отдельный параметр ширины в используемой формуле не вводится: коэффициент при квадрате расстояния равен единице. Для двоичных входов сумма квадратов разностей совпадает с числом несовпадающих координат. Поэтому возможны значения $1, e^{-1}, e^{-2}, e^{-3}, e^{-4}$; в центре функция равна единице.

Сеть имеет четыре входа, семь радиально-базисных нейронов и один выходной нейрон. Центры фиксированы и не обучаются. Настраиваются только восемь параметров выходного слоя: семь весов $v_1, \ldots, v_7$ и смещение v_0.

2.2 Выход сети

Вектор признаков с постоянным входом смещения записывается как

$$\mathbf{\Phi}(\mathbf{x}) = (1, \varphi_1(\mathbf{x}), \ldots, \varphi_7(\mathbf{x})).$$

Выходной нейрон вычисляет взвешенную сумму

$$net(\mathbf{x}) = v_0 + \sum_{j=1}^{7} v_j \varphi_j(\mathbf{x}) \tag{4}$$

и пороговый выход

$$y(\mathbf{x}) = \begin{cases} 1, & net(\mathbf{x}) \geq 0, \\ 0, & net(\mathbf{x}) < 0. \end{cases} \tag{5}$$

Условие при равенстве нулю существенно: при всех нулевых начальных весах сеть выдаёт единицу для любого входа.

3 Алгоритм обучения и поиска выборки

3.1 Коррекция выходных весов

Начальные веса принимаются равными нулю, скорость обучения $\eta = 0{,}3$, предельное число эпох — 1000. Для каждого предъявленного образца сначала вычисляются признаки и выход, затем ошибка бинарного ответа

$$\delta = t - y. \tag{6}$$

После этого выполняются обновления

$$v_j \leftarrow v_j + \eta\delta\varphi_j(\mathbf{x}), \qquad j = 1, \ldots, 7, \tag{7}$$

$$v_0 \leftarrow v_0 + \eta\delta. \tag{8}$$

При $\delta = 0$ параметры не изменяются. Если требовалась единица, но получен ноль, веса увеличиваются в направлении вектора признаков; при противоположной ошибке уменьшаются.

В реализации используется именно разность $t - y$ с пороговым выходом, а не разность $t - net$. Это персептронное правило коррекции выходного слоя. Его следует отличать от классического линейного правила Видроу–Хоффа, минимизирующего квадрат ошибки непрерывной суммы net. Производная пороговой функции при таком обучении не вычисляется; алгоритм Back Propagation не используется.

Одна эпоха — полный проход по выбранной обучающей выборке в фиксированном порядке. После эпохи заново вычисляются ответы на всей таблице при текущих весах и определяется

$$E(k) = \sum_{p=1}^{16} \bigl(t_p - y_p(k)\bigr)^2. \tag{9}$$

Поскольку $t_p, y_p \in \{0, 1\}$, величина $E(k)$ равна числу неверных ответов. Обучение успешно завершено при $E(k) = 0$.

Дополнительно предусмотрено прекращение бесперспективного запуска: если за целую эпоху не было ни одного изменения весов, повторение той же последовательности образцов не изменит состояние сети. Если при этом $E(k) > 0$, выбранное подмножество отклоняется. Запуск также ограничен 1000 эпохами.

3.2 Перебор обучающих подмножеств

Центры и матрица признаков всех 16 входов сохраняются неизменными. Для каждого размера $q = 1, 2, \ldots, 16$ перебираются все сочетания из q различных строк полной таблицы. Каждый кандидат обучается с нулевых выходных весов; внутри кандидата строки предъявляются в возрастающем порядке номеров.

Кандидат считается подходящим только тогда, когда обученная на нём сеть правильно воспроизводит всю таблицу истинности. Перебор завершается на первом подходящем кандидате минимального размера. Таким образом, проверяется минимальность для заданных центров, начальных весов, порядка предъявления, правила обновления и ограничения числа эпох.

Число образцов, используемых для коррекции выходных весов, не равно количеству использованной информации о функции: все 16 меток уже участвовали в выборе центров и используются для проверки. Найденное подмножество не является доказательством обучения неизвестной функции по трём независимым примерам.

4 Результаты численного эксперимента

4.1 Первое обновление

При $\mathbf{v}(0) = \mathbf{0}$ все ответы сети равны единице. Семь нулевых значений функции воспроизводятся неверно, поэтому $E(0) = 7$.

При обучении на полной выборке сначала предъявляется 0000 с целью $t = 1$. Получаем $net = 0$, $y = 1$, $\delta = 0$; изменения отсутствуют. Для следующего образца 0001 целевое значение равно нулю, а выход по-прежнему равен единице. Поэтому $\delta = -1$.

Вектор радиально-базисных признаков этого образца равен

$$(\varphi_1, \ldots, \varphi_7) = (1, e^{-2}, e^{-1}, e^{-2}, e^{-1}, e^{-3}, e^{-2}).$$

После первого ненулевого обновления получаем

$$v_0 = -0{,}3,$$

$$(v_1, v_2, v_3, v_4) \approx (-0{,}300000; -0{,}040601; -0{,}110364; -0{,}040601),$$

$$(v_5, v_6, v_7) \approx (-0{,}110364; -0{,}014936; -0{,}040601).$$

Далее остальные строки выборки последовательно обрабатываются по тому же правилу.

4.2 Обучение на полной таблице

Для каждой эпохи предъявляются 16 образцов. Получены ошибки

$$E(0), \ldots, E(4) = (7, 7, 1, 1, 0).$$

Полная таблица воспроизведена после четырёх эпох, то есть после 64 предъявлений образцов. Состояния весов показаны в таблицах 3 и 4; две строки с одинаковым k вместе задают все восемь параметров.

Таблица 3 – Веса при обучении на полной выборке: смещение и первые три связи

k	v_0	v_1	v_2	v_3
0	0.000000	0.000000	0.000000	0.000000

k	v_0	v_1	v_2	v_3
1	0.300000	-0.174700	-0.010728	-0.064269
2	0.300000	-0.434099	-0.106156	-0.194802
3	0.600000	-0.513372	-0.056563	-0.223965
4	0.600000	-0.443609	-0.341626	-0.293728

Таблица 4 – Веса при обучении на полной выборке: оставшиеся четыре связи

k	v_4	v_5	v_6	v_7
0	0.000000	0.000000	0.000000	0.000000
1	-0.174700	-0.124590	-0.064269	-0.045834
2	-0.174700	-0.220018	-0.323668	-0.176368
3	-0.349400	-0.249181	-0.387937	-0.187096
4	-0.375065	-0.223516	-0.492806	-0.212761

4.3 Минимальная найденная выборка

Перебор проверил 282 кандидата: 16 одноэлементных, 120 двухэлементных и 146 трёхэлементных. Среди подмножеств размеров 1 и 2 успешных запусков не было. Первый успешный кандидат приведён в таблице 5.

Таблица 5 – Минимальная найденная выборка для обучения выходных весов

№ строки	x_1	x_2	x_3	x_4	t
2	0	0	0	1	0
6	0	1	0	1	1
11	1	0	1	0	0

Размер найденного подмножества равен трём; образцы предъявляются в порядке 0001, 0101, 1010. Начальные выходные веса снова нулевые. Ошибка на всей таблице меняется следующим образом:

$$E(0), \ldots, E(4) = (7, 9, 9, 6, 0).$$

Таким образом, и в этом случае потребовалось четыре эпохи, но число предъявлений в успешном запуске составляет $4 \cdot 3 = 12$. Рост ошибки на первых эпохах допустим: коррекция по выбранному образцу не гарантирует монотонного уменьшения ошибки на всей таблице.

В таблицах 6 и 7 приведены все состояния выходных весов при обучении на найденном подмножестве.

Таблица 6 – Веса при обучении на минимальной выборке: смещение и первые три связи

k	v_0	v_1	v_2	v_3
0	0.000000	0.000000	0.000000	0.000000
1	0.000000	-0.189636	-0.025664	-0.069763
2	0.000000	-0.094208	-0.121092	-0.069763
3	0.300000	0.016155	-0.106156	-0.029163
4	0.300000	-0.173481	-0.131821	-0.098926

Таблица 7 – Веса при обучении на минимальной выборке: оставшиеся четыре связи

k	v_4	v_5	v_6	v_7
0	0.000000	0.000000	0.000000	0.000000
1	-0.025664	-0.069763	-0.009441	-0.025664
2	-0.121092	-0.069763	-0.303947	-0.121092
3	-0.106156	-0.029163	-0.298452	-0.106156
4	-0.131821	-0.098926	-0.307893	-0.131821

4.4 Выходные векторы и сравнение экспериментов

В таблице 8 приведены ответы на всех 16 комбинациях после каждой эпохи. Символы в каждой строке вектора идут в порядке образцов таблицы 1, от 0000 до 1111. Целевой вектор неизменен:

$$\mathbf{t} = (1, 0, 0, 0, 1, 1, 1, 1, 0, 0, 0, 0, 1, 1, 1, 1).$$

Таблица 8 – Ответы и ошибки на полной таблице после каждой эпохи

k	Полная выборка	E_{full}	Минимальная выборка	E_{min}
0	1111111111111111	7	1111111111111111	7
1	1111111111111111	7	0000000000000000	9
2	0000111100001111	1	0000000000000000	9
3	1010111100001111	1	1111111111011111	6
4	1000111100001111	0	1000111100001111	0

График на рисунке 1 показывает число неверных ответов на полной таблице для обоих способов обучения.

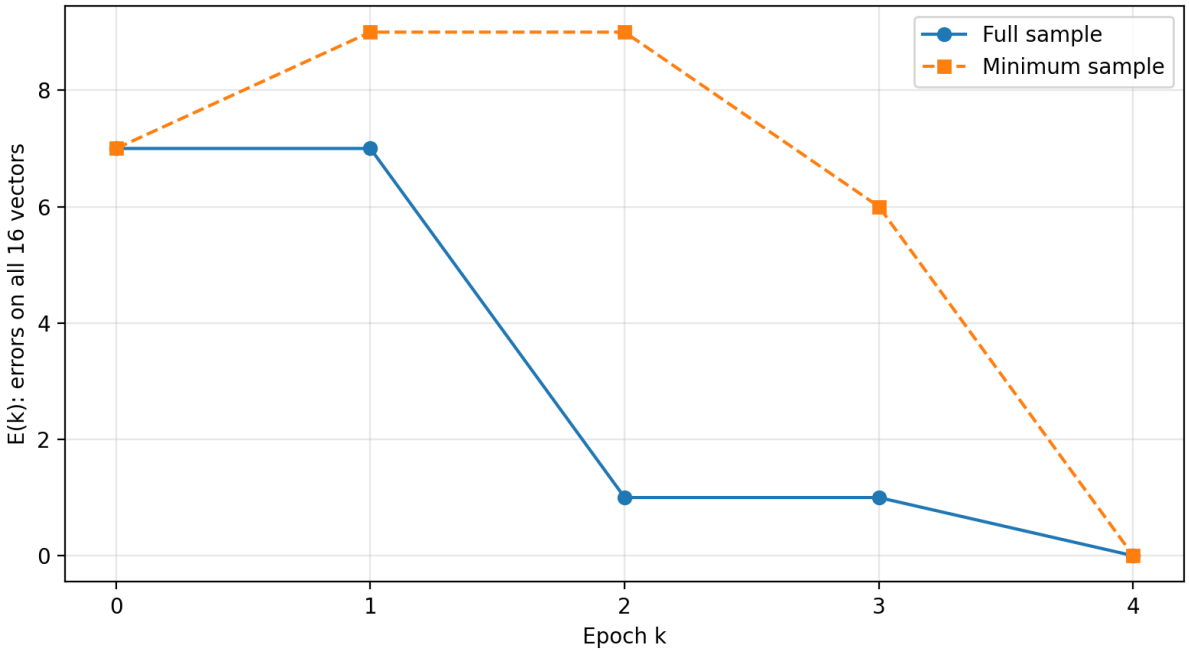


Рисунок 1 – Ошибка на полной таблице истинности по эпохам

Таблица 9 – Сравнение успешных запусков

Показатель	Полная выборка	Минимальная выборка
Обучающих образцов	16	3
Фиксированных центров	7	7
Настраиваемых параметров	8	8
Эпох до $E = 0$	4	4
Предъявлений за успешный запуск	64	12
Верных ответов на полной таблице	16 из 16	16 из 16

Снижение числа предъявлений с 64 до 12 относится только к двум успешным запускам. Оно не учитывает затраты на перебор 282 кандидатов и первоначальное формирование центров по полной таблице. Поэтому поиск минимального подмножества не следует автоматически считать более быстрым способом решения задачи целиком.

5 Программная реализация

Реализация выполнена на Python 3. Полный исходный код `bas_rbf_complete.py` приведён в приложении А, листинг А.1. Для вычислений используются стандартные модули; Matplotlib требуется только для сохранения графика.

Функция `make_truth_table` строит таблицу заданной булевой функции. Функция `choose_centers` выбирает меньший класс, `radial_basis` вычисляет гауссовы функции, а `features` добавляет единичный вход смещения.

Функция `teach_model` выполняет пообразцовую коррекцию весов и после каждой эпохи проверяет все 16 входов. В истории сохраняются номер эпохи, число неверных ответов, копия весов и полный выходной вектор. Копирование весов необходимо, чтобы последующие обновления не изменяли сохранённые состояния предыдущих эпох.

Функция `find_minimum_sample` перебирает сочетания по возрастанию размера. Для каждого кандидата выполняется независимое обучение с нулевых выходных весов. Центры при этом остаются теми же, что и в первом эксперименте.

Программа выводит результаты в консоль и сохраняет `rbf_history.csv`, `rbf_min_history.csv` и `rbf_error.png`. График и числовые данные включены в проект. Для запуска используется команда `python bas_rbf_complete.py`; для установки графической библиотеки — `python -m pip install matplotlib`.

6 Выводы

Реализована радиально-базисная нейронная сеть для булевой функции варианта 2. По меньшему классу таблицы истинности выбраны семь фиксированных центров. Обучение восьми параметров выходного слоя осуществляется по ошибке порогового ответа с коэффициентом $\eta = 0{,}3$.

При обучении на всех 16 образцах полное совпадение с таблицей истинности достигнуто за четыре эпохи. Найдено подмножество из трёх образцов 0001, 0101, 1010, обучение на котором также даёт правильные ответы на всех 16 комбинациях за четыре эпохи.

Минимальность относится к проведённому перебору при фиксированных центрах, нулевых начальных весах, заданном порядке обучения и ограничении в 1000 эпох. Центры и проверка используют полную таблицу функции; независимая оценка обобщения на неизвестной функции не выполнялась.

ПРИЛОЖЕНИЕ А

Исходный код программы

Листинг А.1 – Радиально-базисная сеть и поиск обучающего подмножества

```python
from itertools import product, combinations
from math import exp
from pathlib import Path
import csv

ETA = 0.3
MAX_EPOCHS = 1000
ROOT = Path(__file__).resolve().parent

def target_function(x):
    x1, x2, x3, x4 = x
    return int((not (x3 or x4) and not x1) or x2)

def make_truth_table():
    return [(x, target_function(x)) for x in product((0, 1), repeat=4)]

def choose_centers(table):
    zeros = [x for x, target in table if target == 0]
    ones = [x for x, target in table if target == 1]
    return zeros if len(zeros) <= len(ones) else ones

def radial_basis(x, center):
    return exp(-sum((xi - ci) ** 2 for xi, ci in zip(x, center)))

def features(x, centers):
    return [1.0] + [radial_basis(x, center) for center in centers]

def output_vector(matrix, weights):
    return [int(sum(w * p for w, p in zip(weights, row)) >= 0) for row in matrix]

def evaluate(matrix, targets, weights):
    outputs = output_vector(matrix, weights)
    error = sum((target - output) ** 2 for target, output in zip(targets, outputs))
    return error, outputs

def teach_model(matrix, targets, indices, eta=ETA, max_epochs=MAX_EPOCHS):
    weights = [0.0] * len(matrix[0])
    error, outputs = evaluate(matrix, targets, weights)
    history = [(0, error, weights.copy(), outputs)]
    if error == 0:
        return weights, history
    for epoch in range(1, max_epochs + 1):
        changed = False
        for index in indices:
            row = matrix[index]
            net = sum(w * p for w, p in zip(weights, row))
            output = int(net >= 0)
            delta = targets[index] - output
            if delta != 0:
                changed = True
                for j in range(1, len(weights)):
                    weights[j] += eta * delta * row[j]
                weights[0] += eta * delta
        error, outputs = evaluate(matrix, targets, weights)
        history.append((epoch, error, weights.copy(), outputs))
        if error == 0 or not changed:
            break
    return weights, history

def find_minimum_sample(matrix, targets):
    tested = 0
    for size in range(1, len(matrix) + 1):
        for indices in combinations(range(len(matrix)), size):
            tested += 1
            weights, history = teach_model(matrix, targets, indices)
            if history[-1][1] == 0:
                return indices, weights, history, tested
    raise RuntimeError("No successful training subset was found")

def save_history(filename, history):
    weight_count = len(history[0][2])
    output_count = len(history[0][3])
    header = ["epoch", "E(k)"] + [f"v{i}" for i in range(weight_count)]
    header += [f"y{i + 1}" for i in range(output_count)]
    with (ROOT / filename).open("w", newline="", encoding="utf-8") as file:
```

```
        writer = csv.writer(file)
        writer.writerow(header)
        for epoch, error, weights, outputs in history:
            writer.writerow([epoch, error, *weights, *outputs])

def print_history(title, history):
    print("\n" + title)
    for epoch, error, weights, outputs in history:
        values = ", ".join(f"{weight:.8f}" for weight in weights)
        print(f"k={epoch}; E={error}; v=({values}); y={outputs}")

def save_plot(full_history, minimum_history):
    import matplotlib.pyplot as plt
    fig, ax = plt.subplots(figsize=(8, 4.5))
    ax.plot([r[0] for r in full_history], [r[1] for r in full_history], "o-", label="Full sample")
    xs = [r[0] for r in minimum_history]
    ys = [r[1] for r in minimum_history]
    ax.plot(xs, ys, "s--", label="Minimum sample")
    ax.set_xlabel("Epoch k")
    ax.set_ylabel("E(k): errors on all 16 vectors")
    ax.set_xticks(sorted({r[0] for r in full_history + minimum_history}))
    ax.grid(True, alpha=0.3)
    ax.legend()
    fig.tight_layout()
    fig.savefig(ROOT / "rbf_error.png", dpi=200)
    plt.close(fig)

def main():
    table = make_truth_table()
    targets = [target for x, target in table]
    centers = choose_centers(table)
    matrix = [features(x, centers) for x, target in table]
    print("RBF: lab 4, variant 2")
    print("F = (not (x3 or x4) and not x1) or x2")
    print(f"J0={targets.count(0)}; J1={targets.count(1)}; J={len(centers)}")
    print(f"Centers: {centers}")
    weights, full_history = teach_model(matrix, targets, range(len(table)))
    if full_history[-1][1] != 0:
        raise RuntimeError("Full-sample training did not converge")
    print_history("Training on all 16 vectors", full_history)
    indices, minimum_weights, minimum_history, tested = find_minimum_sample(matrix, targets)
    print(f"\nMinimum sample size: {len(indices)}; tested subsets: {tested}")
    for index in indices:
        print(f"Row {index + 1}: x={table[index][0]}; t={targets[index]}")
    print_history("Training on the minimum sample", minimum_history)
    print("\nFull truth-table validation")
    for i, (x, target) in enumerate(table):
        first = full_history[-1][3][i]
        second = minimum_history[-1][3][i]
        print(f"x={x}; t={target}; y_full={first}; y_minimum={second}")
    save_history("rbf_history.csv", full_history)
    save_history("rbf_min_history.csv", minimum_history)
    save_plot(full_history, minimum_history)

if __name__ == "__main__":
    main()
```